

Задание «MyDrive: многопользовательское файловое хранилище, протокол TCP.»

- Загружать решения в виде ZIP архива в проект Smart LMS под названием HW4
- В архиве должна быть папка, которая включает в себя проект, отчет¹ (DOCX/PDF формат)
- Название ZIP архива должно совпадать с Вашей фамилией.

Мотивация. Для углубленного изучения сетевых шаблонов проектирования (networking design patterns), асинхронности отправки/получения данных и протокола TCP необходима практика в виде разработки многопользовательского многопоточного клиент-серверного приложения.

На лекциях и семинарах рассматривался Netty Framework, но многие сетевые шаблоны проектирования (networking design patterns) – общие для разных сетевых программных Frameworks. Понимание Netty Framework важно, поскольку затрагивает и проясняет много сетевых особенностей на достаточно глубоком уровне для программных инженеров.

Язык программирования и сетевой Framework.

В работе можно использовать один из следующих вариантов:

- Java / Netty Framework (оценка до 10 баллов)
- Rust / Actix-Web / Actix-Net (оценка до 10 баллов если без await)
- C++ / Boost.Asio (оценка до 10 баллов если без coroutines)
- Kotlin / Ktor (не рекомендуются, оценка до 9 баллов²)
- Go / net.Conn / bufio (не рекомендуются, оценка до 9 баллов²)

Python использовать запрещено: при разработке программного приложения слишком много важных принципов сетевой коммуникации скрыто от разработчика.

Оценивание задания:

- код будет анализироваться на корректность и точность выполнения требований этого задания
- будет выполнен анализ полученных результатов и отчета
- будет учитываться качество кода (форматирование, обработка corner cases, иные стандартные элементы качества кода)

Важно: проект необходимо будет защитить во время семинара (оценка выставляется после защиты). Вас могут попросить:

- пояснить код,
- внести изменения в код,
- продемонстрировать работу проекта при разных параметрах и условиях,
- ответить на теоретические вопросы по курсу компьютерных сетей

Расписание защит уточняйте у преподавателей семинарских занятий; преподаватель может применить понижающий коэффициент при пропуске даты защиты.

¹ Примечание: без заполнения ключевых разделов отчета, напр. об использовании ИИ, работа может быть оценена с коэффициентом 0.5 (т. е., по умолчанию будет считаться, что не были выполнены ключевые требования задания либо был использован ИИ, шаблон отчета находится в конце этого файла)

² Kotlin использует coroutines, Go использует goroutines: хотя на практике код выглядит проще, в образовательных целях это препятствует углубленному пониманию асинхронности потоков данных в компьютерных сетях и возникающих при этом проблем.

Содержание задания.

Сценарий использования: клиентское приложение запускается на устройстве пользователя и синхронизирует с сервером заданную директорию по протоколу TCP (непрямые аналоги – см. Яндекс Диск, OneDrive, и подобные приложения).

Пусть **БО** – базовая оценка, до 5 либо 6 баллов (в зависимости от выбранного языка и Framework, где максимальная оценка до 9 либо 10 баллов, соответственно).

Словесное описание бизнес протокола (custom protocol), базовая функциональность на оценку **БО**:

- клиент подключается к серверу по протоколу TCP
- сообщает свой ID (это не авторизация, просто чтобы отличать пользователей друг от друга), от запуска к запуску на одном и том же устройстве должен быть один и тот же ID
- затем клиент сообщает серверу список файлов (поддиректории необязательно обрабатывать) – для каждого файла имя, размер, контрольные суммы (в одном сообщении клиент -> сервер)
- сервер в ответ сообщает клиенту (в одном сообщении сервер -> клиент) какие файлы отсутствуют либо их контрольные суммы не совпадают с полученными
- клиент отправляет серверу все файлы, которых нет на сервере либо чьи контрольные суммы не совпадают с имеющимися на сервере
- клиент может отправлять одновременно до 1 файла, по протоколу TCP передает файлы серверу
- сервер обязан поддерживать неограниченное количество подключаемых к нему клиентов (под неограниченным количеством подразумеваем факт, при котором мы со своей стороны специально не контролируем и не ограничиваем подключения, их может контролировать Framework или операционная система)
- размеры передаваемых файлов – до 1 ГБ; для демонстрации работы подготовить не менее 10 файлов каждый размером не менее 100 МБ
- после завершения раунда коммуникации клиента и сервера, на сервере для указанного ID пользователя в заданной директории (путь задается при старте в командной строке либо в конфигурационном файле) находится точная копия файлов из директории клиента

Параллельная отправка нескольких файлов, **до +2 баллов**:

- для отправки файлов клиент создает несколько соединений к серверу (ограничения см. ниже) и отправляет серверу несколько файлов параллельно (одновременно) в нескольких соединениях
- сервер допускает установку неограниченного количества соединений от неограниченного количества клиентских приложений (неограниченное количество соединений от одного и того же клиентского приложения и неограниченное количество клиентов)

Передача файлов с использованием DMA (Direct Memory Access), **до +2 баллов**

- используем DMA, не копируем содержимое файла в память для передачи буфера из памяти. Если Вы использовали DMA, то клиент должен уметь работать в двух режимах – с DMA и без DMA, чтобы замерить время передачи в обоих случаях для сравнения (см. шаблон отчета). В Netty для использования DMA есть `DefaultFileRegion`; в других Frameworks есть свои аналоги. Если вы передаете только один файл одновременно, то за использование DMA можно получить **до +1 балла**. Если Вы реализовали отправку нескольких файлов одновременно (см. выше), то каждый файл должен отправляться с помощью DMA, чтобы получить до +2 баллов.

У клиента должен быть конфигурационный файл с ID, путем к директории, IP адресом и портом сервера, максимальным числом подключений к серверу (поддерживать от 1 до 32 одновременных подключений).

Можно использовать любой Framework для сериализации / десериализации сообщений, но обязательно реализовать шаблон коммуникации (communication pattern) на слайде №14 лекции №9 (файл `networks-course-2026-09-tcp-part01`). При этом в результате сериализации Вы должны получить массив байт, который явным образом отправляется по протоколу TCP (отправляем не абстрактное сообщение с помощью высокоуровневого Framework, а байты). Аналогично Вы должны получать массив байт по частям от протокола TCP и затем десериализировать из него сообщение (клиента на стороне сервера либо сервера на стороне клиента).

Для реализации шаблона коммуникации рекомендуется использовать

Java: `ReplayingDecoder` либо `ByteToMessageDecoder`

Rust: `actix-codec + Decoder trait`

Kotlin: `ByteReadChannel`

C++: `async_read`

Обратите внимание, что работа с помощью DMA проходит несколько иным образом, нежели передача информационных сообщений. Без DMA файл можно передать в описанном выше базовом протоколе, при использовании DMA – иначе, не сериализацией / десериализацией объектов (проектирование подхода является частью этого задания).

Вся логика должна быть запрограммирована на уровне выбранного Framework.

Например, нельзя перейти на Spring WebFlux и использовать `endpoints`, `annotations`, хоть и Netty Framework будет «под капотом».

Клиентское приложение может не сканировать периодически появление новых файлов в директории (хотя это может быть преимуществом при недочетах в других частях реализации). Напротив, можно в командной строке в консоли реализовать считывание команды для активации повторного сканирования директории и ее синхронизации с сервером.

Шаблон отчета

ОТЧЕТ ПО HW4

ФИО, группа, email

Цель работы: *укажите цель работы своими словами*

Я претендую на оценку до 5 баллов, был использован генеративный ИИ (далее следует информация из декларации согласно https://www.hse.ru/studyspravka/ai_guidelines/)

ЛИБО

Я претендую на оценку до 9 (10) баллов (зависит от выбранных языка программирования и сетевого Framework), генеративный ИИ не был использован для работы над этим заданием.

Укажите выбранные язык программирования и сетевой Framework.
Кратко опишите архитектуру решения.

Приведите ключевые фрагменты кода (основные классы) реализации бизнес протокола коммуникации между клиентом и сервером. Если использовался сторонний Framework сериализации, опишите шаги его использования.

Далее необходимо привести факты, свидетельствующие о том, что Ваше приложение работало (клиент и сервер взаимодействовали):

- Скриншоты WireShark
- Какие файлы передавались (в том числе их размеры)
- Примеры одновременной передачи нескольких файлов серверу
- иные факты

В отчет включите наблюдения по поведению протокола TCP (скриншоты из WireShark и вывод других утилит):

- какие значения MSS и Window Вы наблюдали в процессе обмена данными
- наблюдался ли медленный старт TCP
- были ли эффекты congestion и каким образом они смягчались
- иные наблюдения

Преподаватель может запросить и другие подтверждения при необходимости, в Ваших интересах привести полную картину экспериментальной установки.

Если Вы использовали DMA, приведите графики (замеры времени) передачи больших файлов разного размера при использовании DMA и без его использования для сравнения (по меньшей мере такие размеры файлов – 50 МБ, 100 МБ, 250 МБ).

- Будьте готовы продемонстрировать работу приложения на семинаре.
- Заранее разверните (deploy) Ваш сервер на удаленной виртуальной машине Linux (преподаватель предоставит доступ и откроет порт в облаке, если он закрыт либо Вы можете развернуть серверное приложение в Облаке своего аккаунта, либо своем VPS, если таковые у Вас имеются).
- Проверьте возможность подключения клиентского приложения на Вашем устройстве (ноутбуке) либо машине в аудитории к своему удаленному серверу и отправьте файлы.
- Будьте готовы продемонстрировать, какие характеристики сетевого взаимодействия можно собрать с помощью утилит командной строки, которые рассматривались на семинарах (напр., ss, tcpdump, tc и другие утилиты).